



HOSTEL RECRUITMENT SYSTEM

Computer Programming CEP



GROUP MEMBERS

NAME: TAYYAB MAHMOOD ADIL	NAME: MUA AZ AHMAD
ROLL NO. : 2502272-B	ROLL NO. : 2502338-B

DECEMBER 20, 2025

SUBMITTES TO:
MA'AM MARYAM SHEIKH

Contents

CEP Report: Hostel Recruitment System	2
1. Abstract	2
2. Introduction	2
2.1 Background.....	2
2.2 Objective.....	2
2.3 Scope.....	2
3. Project Fundamentals.....	2
4. Part-wise Code Explanation.....	3
Part 1: Libraries and Data Structure	3
Part 2: Helper Functions & Initialization.....	5
Part 3: The Main Loop & Menu.....	6
.....	6
.....	6
Part 4: Display & Selection Logic	7
.....	7
.....	7
.....	8
Part 5: Room Configuration & Cost Logic	8
Part 6: The Receipt.....	9
5. Final Results	10
6. Conclusion	12

CEP Report: Hostel Recruitment System

1. Abstract

The **Hostel Recruitment System** is a console-based application developed in C++ designed to simplify the process of finding and booking hostel accommodation for students. In busy educational hubs like Islamabad, finding suitable housing with specific amenities (mess, attached washrooms, room occupancy) can be challenging. This program acts as a digital directory and a cost estimator. It maintains a database of boys' and girls' hostels, allows users to filter by category, customize their room preferences, and generates an immediate estimated monthly receipt. The project utilizes fundamental programming concepts such as Structures (struct), Arrays, Loops, and Input Validation to ensure a smooth user experience.

2. Introduction

2.1 Background

For students (also for job persons) moving to a new city for education, accommodation is the first major hurdle. The traditional method involves visiting multiple hostels physically to check availability and ask about prices. This is time-consuming and inefficient.

2.2 Objective

The primary objective of this project is to create a user-friendly interface that:

1. **Centralizes Information:** Stores details (Name, Address, Contact, and Location Ratings) of various hostels in one place.
2. **Categorizes Data:** Separates hostels into "Boys" and "Girls" categories for easier navigation.
3. **Calculates Costs:** Provides an accurate financial estimate based on user choices (Room type, Food/Mess inclusion, and Washroom preferences).
4. **Generates Receipts:** outputs a clear breakdown of the final monthly rent.

2.3 Scope

Currently, the system is designed for the Islamabad region, containing a pre-populated list of 20 hostels (10 for boys and 10 for girls). It is a scalable system where more data can be added via the source code.

- **Scalability:**

The code is written modularly, meaning new cities or hostels can be added by simply calling the add Hostel function.

3. Project Fundamentals

The core logic of this program operates on an **Input-Process-Output** model. Here is the simple fundamental workflow:

1. **Data Structure (The "Database"):** Instead of using external files or SQL, the program uses **Structures** and **Arrays**. A Structure holds the details of a each hostel, and an Array holds the list of all hostels. This acts as our temporary database.
2. **Menu Logic:** The program runs inside a while loop. This keeps the application running until the user specifically chooses to "Exit," preventing the console from closing after just one action.
3. **Selection & Customization:**
 - **Step 1:** User selects Gender (Boys/Girls).
 - **Step 2:** User selects a specific Hostel from the list.
 - **Step 3:** User selects Room Type (1, 2, 3, or 4 seater). Prices drop as the room capacity increases.
 - **Step 4:** User toggles "Add-ons" (Mess and Attached Washroom) using y/n inputs.

4. The Math Engine:

The total cost is calculated using this formula:

Total = Room Price + Washroom Cost + Mess Cost + Luxury

5. Validation:

To prevent the program from crashing if a user types "abc" instead of "1", the program reads inputs as strings first, then converts or compares them.

4. Part-wise Code Explanation

Part 1: Libraries and Data Structure

```
#include <iostream>
#include <string>
using namespace std;

struct Hostel {
    string name;
    string address;
    string contact;
    int luxuryCharge;
};

Hostel boysHostels[10];
Hostel girlsHostels[10];
```

Libraries:

- <iostream>: Essential for cin (input) and cout (output).
- <string>: Used instead of char[] arrays.
- using namespace std::: allows us to use cin and cout without std ::

Struct Hostel:

This defines the "shape" of our data. It encapsulates four distinct attributes for every hostel.

- *The luxury Charge Logic*: This integer field allows the system to model real-world pricing variance. A hostel in a posh sector (like F-8) can carry a hidden surcharge compared to one in a remote area, making the pricing model more realistic.

Global Arrays: Defined outside main(), these arrays act as the primary storage containers accessible by all functions in the program.

Part 2: Helper Functions & Initialization

```

void addHostel(Hostel list[], int index, string n, string a, string c, int l) {
    list[index].name = n;
    list[index].address = a;
    list[index].contact = c;
    list[index].luxuryCharge = l;
}

void initializeData() {

    addHostel(boysHostels, 0, "Ibrahim Hostel & Guest House", "Street 5, G-9/1, Islamabad", "0300-1234567", 0);
    addHostel(boysHostels, 1, "Punjab Boys Hostel Islamabad", "H-13, Near NUST, Islamabad", "0321-9876543", 500);
    addHostel(boysHostels, 2, "Royal Blue Boys Hostel", "F-8/2, Islamabad", "0333-5556667", 2000);
    addHostel(boysHostels, 3, "Azam Boys Hostel", "G-10 Markaz, Islamabad", "0312-4443332", 0);
    addHostel(boysHostels, 4, "The Luxurious Boys Hostel", "E-11/3 MPCHS, Islamabad", "0301-1112223", 4000);
    addHostel(boysHostels, 5, "Khan Boys Hostel", "G-11/2, Islamabad", "0345-7778889", 0);
    addHostel(boysHostels, 6, "I-8/4 Boys Hostels Shaheen", "I-8/4, Near Metro, Islamabad", "0302-3334445", 1500);
    addHostel(boysHostels, 7, "G-10 Boys Hostel Shaheen", "G-10/4, Islamabad", "0322-2223334", 1000);
    addHostel(boysHostels, 8, "Lavish Group of Boys Hostel", "Blue Area, Islamabad", "0334-9998887", 3000);
    addHostel(boysHostels, 9, "Awans Boys Hostel", "Soan Garden, Islamabad", "0315-6667778", 0);

    addHostel(girlsHostels, 0, "Falah Girls Hostel", "G-9/4, Islamabad", "0333-1122334", 0);
    addHostel(girlsHostels, 1, "Taaaj Girls Hostel", "F-7/2, Islamabad", "0321-4433221", 2500);
    addHostel(girlsHostels, 2, "Al Muqtadir Girls Hostel", "I-10/2, Islamabad", "0300-5554443", 500);
    addHostel(girlsHostels, 3, "I-8/2 Girls Hostel Shaheen", "I-8/2, Islamabad", "0345-9990001", 1500);
    addHostel(girlsHostels, 4, "Pak Home Girls Hostel", "G-11/3, Islamabad", "0312-3332221", 1000);
    addHostel(girlsHostels, 5, "Fairy Land Hostel", "E-11/2, Islamabad", "0301-7776665", 3500);
    addHostel(girlsHostels, 6, "Grace Female Hostel", "F-10/3, Islamabad", "0332-1110009", 2000);
    addHostel(girlsHostels, 7, "Nazzal Islamabad Girls Hostel", "G-13/1, Islamabad", "0323-5556667", 1000);
    addHostel(girlsHostels, 8, "Palz Care Hostel", "H-13, Near University, Isb", "0305-4445556", 0);
    addHostel(girlsHostels, 9, "Alkhairi Girls Hostel", "PWD Housing Society, Isb", "0313-8889990", 0);

}

void displayHostels(Hostel list[], int size) {
    for (int i = 0; i < size; i++) {
        cout << "\t\t" << (i + 1) << ". " << list[i].name << endl;
    }
}

```

- **addHostel:** This function saves us from writing 4 lines of code for every hostel. We pass the data in one line, and this function assigns it to the structure.
- **initializeData:** This function is called immediately when the program starts. It "hardcodes" the data into the arrays so the lists aren't empty when the user runs the app.
- **displayHostels:** This function is called when the user input his/her required criteria and it displays the specific list of hostels according to gender criteria.

Part 3: The Main Loop & Menu

```
int main() {
    initializeData();
    string inputStr;
    int mainChoice = 0;
    int hostelIndex = 0;
    bool running = true;

    while (running) {

        for(int i=0; i<5; i++) cout << endl;

        cout << "\t\t*****\n";
        cout << "\t\t* HOSTEL RECRUITMENT SYSTEM * \n";
        cout << "\t\t*****\n";

        cout << "\t\t\t\t\t MAIN MENU\n";
        cout << "\t\t\t\t\t===== \n";
        cout << "\t\t\t [1] Boys Hostels\n";
        cout << "\t\t\t [2] Girls Hostels\n";
        cout << "\t\t\t [3] Exit System\n";
        cout << "\t\t\t\t\t===== \n";
        cout << "\t\t\t Enter Choice: ";

        cin >> inputStr;
```

```
cin >> inputStr;
mainChoice = 0;

if (inputStr == "1") mainChoice = 1;
else if (inputStr == "2") mainChoice = 2;
else if (inputStr == "3") mainChoice = 3;

if (mainChoice == 0) {
    cout << "\t\t Invalid Choice! Press Enter to try again.";
    cin.ignore(); cin.get();
    continue;
}

if (mainChoice == 3) {
    cout << "\n\t\t Exiting Application... Thank you!\n";
    running = false;
    break;
}
```


- **initializeData():** The first thing main does, is load the database.
- **while(running):** This creates the application loop. The program will keep coming back to the main menu after a booking is finished, unless running becomes false.
- **String Input:** Notice `cin >> inputStr`. We read the menu choice as text. This is a safety feature. If we used `int`, and the user typed "A", the program would crash. By reading a string, we can safely check if (`inputStr == "1"`).

Part 4: Display & Selection Logic

```
Hostel* selectedList = NULL;
string category = "";

if (mainChoice == 1) {
    selectedList = boysHostels;
    category = "BOYS";
} else {
    selectedList = girlsHostels;
    category = "GIRLS";
}

cout << "\n\n";
cout << "\t\t===== \n";
cout << "\t\t LIST OF " << category << " HOSTELS \n";
cout << "\t\t===== \n";

displayHostels(selectedList, 10);

cout << "\n\t\t Select a Hostel (1-10): ";
cin >> inputStr;
```

```
hostelIndex = 0;

if (inputStr == "1") hostelIndex = 1;
else if (inputStr == "2") hostelIndex = 2;
else if (inputStr == "3") hostelIndex = 3;
else if (inputStr == "4") hostelIndex = 4;
else if (inputStr == "5") hostelIndex = 5;
else if (inputStr == "6") hostelIndex = 6;
else if (inputStr == "7") hostelIndex = 7;
else if (inputStr == "8") hostelIndex = 8;
else if (inputStr == "9") hostelIndex = 9;
else if (inputStr == "10") hostelIndex = 10;

if (hostelIndex < 1 || hostelIndex > 10) {
    cout << "\t\t Invalid Selection! Returning to Menu...";
    cin.ignore();
    cin.get();
    continue;
}

Hostel selectedHostel = selectedList[hostelIndex - 1];
```



```

cout << "\n\n";
cout << "\t\t===== \n";
cout << "\t\t HOSTEL DETAILS \n";
cout << "\t\t===== \n";
cout << " Name:   " << selectedHostel.name << endl;
cout << " Address: " << selectedHostel.address << endl;
cout << " Contact: " << selectedHostel.contact << endl;
cout << "\t\t===== \n";

```

- **Pointers (Hostel* selectedList):** Instead of writing two separate blocks of code (one for boys, one for girls), we use a pointer. If the user picks '1', the pointer points to the Boys Array. If '2', it points to the Girls Array. This reduces code duplication.
- **displayHostels:** A loop prints the names stored in the selected array.

Part 5: Room Configuration & Cost Logic

```

int seaterChoice = 0;
string washroomInput, messInput;

cout << "\n\t\t ROOM CONFIGURATION \n";
cout << "\t\t ----- \n";
cout << "\t\t [1] Single Seater (Private) \n";
cout << "\t\t [2] Bi-Seater   (2 Persons) \n";
cout << "\t\t [3] Tri-Seater   (3 Persons) \n";
cout << "\t\t [4] Tetra-Seater (4 Persons) \n";
cout << "\t\t Select Room Type: ";
cin >> inputStr;

if (inputStr == "1") seaterChoice = 1;
else if (inputStr == "2") seaterChoice = 2;
else if (inputStr == "3") seaterChoice = 3;
else if (inputStr == "4") seaterChoice = 4;
else {
    cout << "\t\t Invalid Room Type! Defaulting to Standard (Bi-Seater). \n";
    seaterChoice = 2;
}

```

```

int roomPrice = 0;
string roomType = "";

if (seaterChoice == 1) { roomPrice = 25000; roomType = "Single Seater"; }
else if (seaterChoice == 2) { roomPrice = 20000; roomType = "Bi-Seater"; }
else if (seaterChoice == 3) { roomPrice = 16000; roomType = "Tri-Seater"; }
else if (seaterChoice == 4) { roomPrice = 13000; roomType = "Tetra-Seater"; }
else { roomPrice = 20000; roomType = "Standard"; }

int washroomCost = 0;
if (washroomInput == "y" || washroomInput == "Y") {
    washroomCost = 3000;
}

int messCost = 0;
if (messInput == "y" || messInput == "Y") {
    messCost = 8000;
}

int totalFare = roomPrice + washroomCost + messCost + selectedHostel.luxuryCharge;

```

```
cout << "\t\t Need Attached Washroom? (y/n): ";
cin >> washroomInput;

while(washroomInput != "y" && washroomInput != "Y" && washroomInput != "n" && washroomInput != "N") {
    cout << "\t\t Invalid! Please enter 'y' or 'n': ";
    cin >> washroomInput;
}

cout << "\t\t Include Mess/Food? (y/n): ";
cin >> messInput;

while(messInput != "y" && messInput != "Y" && messInput != "n" && messInput != "N") {
    cout << "\t\t Invalid! Please enter 'y' or 'n': ";
    cin >> messInput;
}
```

- This section handles the business logic.
- It assigns a base price depending on privacy (Single vs Tetra).
- It adds a fixed rate for Washroom (3000) and Mess (8000) if the user inputs 'y'.
- Finally, it adds the specific `luxuryCharge` associated with the specific hostel chosen (defined in Part 2).

Part 6: The Receipt

[illegible]

- The **final block** prints a summary to the console, showing the user exactly what they booked and how much it will cost per month.

5. Final Results

```
*****
* HOSTEL RECRUITMENT SYSTEM *
*****
MAIN MENU
=====
[1] Boys Hostels
[2] Girls Hostels
[3] Exit System
=====
Enter Choice: 1
```

Step 1: The Main Menu

This is the starting point of the application.

- **System Display:** The program displays the title "**HOSTEL RECRUITMENT SYSTEM**" and offers a Main Menu with three options:
 1. Boys Hostels
 2. Girls Hostels
 3. Exit System
- **User Action:** The user is asked to Enter Choice. In this image, the user has typed **1** to select the category for **Boys Hostels**.

```
=====
LIST OF BOYS HOSTELS
=====
1. Ibrahim Hostel & Guest House
2. Punjab Boys Hostel Islamabad
3. Royal Blue Boys Hostel
4. Azam Boys Hostel
5. The Luxurious Boys Hostel
6. Khan Boys Hostel
7. I-8/4 Boys Hostels Shaheen
8. G-10 Boys Hostel Shaheen
9. Lavish Group of Boys Hostel
10. Awans Boys Hostel

Select a Hostel (1-10): |
```

Step 2: Hostel Selection List

Because the user selected "Boys Hostels" in the previous step, the program now filters and displays the relevant list.

- **System Display:** A list of 10 different boys' hostels is shown (e.g., Ibrahim Hostel, Punjab Boys Hostel, etc.).
- **User Action:** The prompt asks the user to Select a Hostel (1-10).

```

=====
HOSTEL DETAILS
=====
Name:    I-8/4 Boys Hostels Shaheen
Address: I-8/4, Near Metro, Islamabad
Contact: 0302-3334445
=====

ROOM CONFIGURATION
=====
[1] Single Seater (Private)
[2] Bi-Seater   (2 Persons)
[3] Tri-Seater  (3 Persons)
[4] Tetra-Seater (4 Persons)
Select Room Type: 3
Need Attached Washroom? (y/n): n
Include Mess/Food? (y/n): y

```

Hostel Details & Room Configuration

The user has now entered the specific booking phase for the hostel selected in Step 2.

- **System Display:** The top section shows the static details for the chosen hostel (Name, Address, and Contact Number).
- **User Action:** The program asks for specific booking preferences:
 1. **Room Type:** The user selected **3** for a "Tri-Seater" (a room for 3 people).
 2. **Washroom:** The user entered **n** (No) for an attached washroom.
 3. **Mess/Food:** The user entered **y** (Yes) to include food services.

```

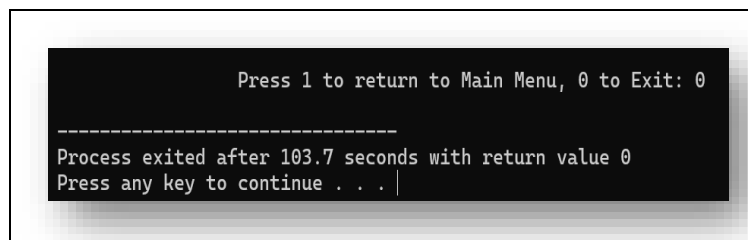
=====
FINAL BOOKING RECEIPT
=====
Hostel:      I-8/4 Boys Hostels Shaheen
Room Type:   Tri-Seater (16000 PKR)
Location Fee: 1500 PKR
Washroom:    Common (0)
Mess:        Included (+8000)
=====
TOTAL MONTHLY FARE: PKR 25500/-
=====

```

Final Booking Receipt

The system has processed the inputs from Step 3 and generated a financial summary.

- **System Display:** It outputs a "FINAL BOOKING RECEIPT" that breaks down the costs:
- **Room Cost:** 16,000 PKR (for the Tri-Seater selected earlier).
- **Location Fee:** 1,500 PKR (a standard fee added by the system).
- **Washroom:** 0 (since the user selected 'No').
- **Mess:** +8,000 (added because the user selected 'Yes').



```

Press 1 to return to Main Menu, 0 to Exit: 0

-----
Process exited after 103.7 seconds with return value 0
Press any key to continue . . . |
  
```

Program Exit

After the transaction or booking display is complete, the program offers a final navigation choice.

- **System Display:** The prompt asks the user to press 1 to return to the Main Menu or 0 to Exit.
- **User Action:** The user entered 0.
- **Result:** The console shows "Process exited after... with return value 0," indicating the program has successfully terminated.

6. Conclusion

The **Hostel Recruitment System** successfully demonstrates how core C++ concepts—such as data structures, pointers, loops, and conditional logic—can be effectively combined to solve a real-world problem. By providing a centralized platform for hostel information and an automated cost calculator, the program significantly improves the efficiency of searching for accommodation compared to manual methods.

While the current implementation relies on in-memory storage (arrays) for simplicity and speed, the modular design allows for future scalability. Potential enhancements could include integrating file handling for persistent data storage, SQL database connectivity for real-time updates, or a Graphical User Interface (GUI) for a more modern user experience. Overall, this project fulfills the goal of creating a functional, error-resistant, and user-friendly console application.
